

revisao_prova_openmp

Revisão para Prova — OpenMP / Programação Concorrente

Cobre: Labs 7-11. Sem gabarito nas questões — resolva primeiro, confira depois na seção final.

Tempo sugerido: ~60-75 min para resolver tudo, mais 20 min revisando o gabarito.

PARTE 1 — Fundamentos (Lab 7)

Q1. Explique a diferença de comportamento entre:

```
#pragma omp parallel
{
    printf("Alo\n");
}
```

e

```
#pragma omp parallel
{
    #pragma omp single
    printf("Alo\n");
}
```

Com `num_threads(4)`, quantas vezes "Alo" é impresso em cada caso?

Q2. Diferencie `master` e `single` em duas frases: quem executa, e se há barreira implícita.

Q3. O código abaixo soma um vetor de forma sequencial. Reescreva-o paralelizado, usando **apenas** `parallel`, `single` / `master` e a cláusula que evita condição de corrida na soma:

```
double soma = 0.0;
for (int i = 0; i < N; i++) {
    soma += A[i];
}
```

PARTE 2 — Schedule (Lab 8)

Q4. Para o laço abaixo, com 4 threads:

```
#pragma omp parallel for num_threads(4) schedule(static, 2)
for (int i = 0; i < 16; i++) { ... }
```

Escreva quais valores de `i` cada thread (0 a 3) vai executar.

Q5. Para o mesmo laço com `schedule(dynamic, 4)`, por que **não** é possível prever com certeza quais iterações cada thread vai pegar, ao contrário do `static`?

Q6. Complete a tabela:

Schedule	Decisão tomada quando?	Melhor cenário de uso
static		
dynamic		
guided		

Q7. Você tem um laço de 1000 iterações onde as 500 primeiras são muito rápidas e as 500 últimas são muito lentas (custo desigual). Qual schedule você escolheria e por quê?

PARTE 3 — Escopo de Variáveis (Lab 8)

Q8. Qual é o escopo **padrão** de uma variável declarada fora de uma região `#pragma omp parallel`, quando nenhuma cláusula de escopo é usada?

Q9. Preveja a saída (ou explique por que é imprevisível):

```
int x = 100;
#pragma omp parallel num_threads(4)
{
    x = x + omp_get_thread_num();
    printf("%d\n", x);
}
printf("final: %d\n", x);
```

Q10. Reescreva o código acima trocando `x` por `private(x)`. O que acontece com o valor de `x` **dentro** da região paralela, e com o valor **fora**, depois da região terminar?

Q11. Diferencie `private` e `firstprivate` com uma frase cada, focando em: **valor inicial herdado ou não**.

Q12. No código abaixo, qual variável está com escopo **incorreto**, causando condição de corrida? Corrija usando a cláusula adequada:

```

int i, j;
long temp_fatorial;
#pragma omp parallel for num_threads(4)
for (i = 1; i <= 8; i++) {
    temp_fatorial = 1;
    for (j = 1; j <= i; j++) {
        temp_fatorial *= j;
    }
    printf("Fatorial de %d = %ld\n", i, temp_fatorial);
}

```

PARTE 4 — Sincronização: critical, atomic, barrier, ordered (Lab 9)

Q13. Classifique cada trecho como `atomic read`, `atomic write`, `atomic update` ou `atomic capture` (ou "não pode ser atomic, precisa de critical"):

- a) `v = contador;`
- b) `contador = 0;`
- c) `contador += 5;`
- d) `resultado = contador++;`
- e) `{ resultado = contador; contador = contador * 2; contador += 1; }`

Q14. Por que a linha (e) da questão anterior não pode virar um único `atomic`?

Q15. Complete: "Use `atomic` quando ___, e use `critical` quando ___."

Q16. No código abaixo, duas regiões `critical` **sem nome** protegem variáveis completamente diferentes (`a` e `b`). Isso é eficiente? Por quê? Como você melhoraria?

```

#pragma omp parallel num_threads(4)
{
    #pragma omp critical
    a++;
    #pragma omp critical
    b++;
}

```

Q17. Explique com suas palavras o que `#pragma omp barrier` faz, e cite uma situação em que você precisaria inserir uma **explicitamente** (não a implícita).

Q18. No código de `ordered` abaixo, o que garante ordem sequencial: o laço inteiro, ou só o bloco marcado?

```
#pragma omp parallel for ordered schedule(static,1)
for (int i = 0; i < 10; i++) {
    printf("A: %d\n", i);          // (1)
    #pragma omp ordered
    { printf("B: %d\n", i); }      // (2)
}
```

Entre a linha (1) e a linha (2), qual delas pode sair em qualquer ordem entre as threads, e qual sempre sai na sequência 0,1,2...9?

PARTE 5 — collapse, nowait, reduction (Lab 8 e 9)

Q19. Por que, no código abaixo, uma das 4 threads fica ociosa (sem trabalho)? Como resolver com uma única cláusula?

```
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 10; j++) { ... }
}
```

Q20. No trecho abaixo, o `nowait` no primeiro laço é seguro ou perigoso? Justifique.

```
#pragma omp parallel num_threads(4) shared(a,b)
{
    #pragma omp for nowait
    for (int i = 0; i < N; i++) a[i] = i * 2;

    #pragma omp for
    for (int i = 0; i < N; i++) b[i] = a[i] + 1;
}
```

Q21. Reescreva o cálculo de PI abaixo com a diretiva e cláusulas corretas (schedule não é obrigatório, mas escopo sim):

```
double soma = 0.0, x;
for (long i = 0; i < num_passos; i++) {
    x = (i + 0.5) * passo;
    soma = soma + 4.0 / (1.0 + x*x);
}
```

PARTE 6 — Tasks: criação, escopo, taskwait/taskgroup (Lab 10 e 11)

Q22. No código abaixo, quantas tasks são criadas ao todo, com `num_threads(4)` ?

```
#pragma omp parallel num_threads(4)
{
    #pragma omp task
    printf("oi\n");
}
```

Q23. Corrija o código da questão anterior para que apenas 1 task seja criada.

Q24. Aplique as regras de escopo de tasks e preencha:

```
int g = 1; // global
void foo() {
    int a = 2, b = 3;
    #pragma omp parallel shared(a) num_threads(2)
    {
        int c = 4;
        #pragma omp single
        {
            #pragma omp task
            {
                int d = 5;
                // g: ?    a: ?    b: ?    c: ?    d: ?
            }
        }
    }
}
```

Q25. No código abaixo, por que `x` e `y` precisam ser `shared`, e não o padrão (`firstprivate`)?

```
int fib(int n) {
    if (n < 2) return n;
    int x, y;
    #pragma omp task shared(x)
    x = fib(n - 1);
    #pragma omp task shared(y)
    y = fib(n - 2);
    #pragma omp taskwait
    return x + y;
}
```

Q26. Diferencie `taskwait` e `taskgroup` numa frase cada. No exemplo abaixo, a mensagem final pode aparecer antes de "Neto terminou"?

```
#pragma omp task
{
    #pragma omp task
    { usleep(200000); printf("Neto terminou\n"); }
    printf("Filho terminou\n");
}
#pragma omp taskwait
printf("Mensagem final\n");
```

Q27. Diferencie `barrier` e `taskwait`: quais threads cada um afeta, e o que cada um espera terminar.

Q28. O que `final(n <= 20)` faz, e por que ele ajuda a resolver o problema de overhead em recursões como Fibonacci com tasks muito pequenas?

Q29. Qual é a diferença prática entre `if(expr)` e `final(expr)` numa task — pensando em quantos níveis da recursão cada cláusula afeta?

PARTE 7 — depend e taskloop (Lab 11)

Q30. Dado o grafo:

```
T1 → T2, T3 → T4
```

Complete as cláusulas `depend` corretas para T1 (produz x), T2 (lê x, produz y), T3 (lê x, produz z), T4 (lê y e z):

```
#pragma omp task depend(_____)
{ x = 10; } // T1

#pragma omp task depend(_____)
{ y = x * 2; } // T2

#pragma omp task depend(_____)
{ z = x + 5; } // T3

#pragma omp task depend(_____)
{ resultado = y + z; } // T4
```

Q31. T2 e T3 da questão anterior podem executar em paralelo entre si? Por quê?

Q32. Se T2 usasse `depend(inout: x)` em vez de `depend(in: x) depend(out: y)`, o que mudaria na relação entre T2 e T3?

Q33. Diferencie `grainsize(n)` e `num_tasks(n)` em `taskloop`.

Q34. Por padrão, `taskloop` tem uma sincronização implícita. Qual é, e o que `nogroup` faz com ela?

Q35. Explique por que a ordem de impressão das iterações de um `taskloop` não é necessariamente crescente, mesmo que o vetor final resultante esteja correto.

PARTE 8 — Questões de código completo (mescla vários tópicos)

Q36. O código abaixo tem 2 erros de paralelização. Identifique e corrija:

```
%%writefile pi_paralelo.c
#include <stdio.h>
#include <omp.h>

int main() {
    long num_passos = 100000000;
    double passo = 1.0 / num_passos;
    double x, pi, soma = 0.0;

    for (long i = 0; i < num_passos; i++) {
        x = (i + 0.5) * passo;
        soma = soma + 4.0 / (1.0 + x*x);
    }

    pi = passo * soma;
    printf("PI = %f\n", pi);
    return 0;
}
```

(Adicione a paralelização correta, escopando `x` e `soma` adequadamente.)

Q37. Escreva um programa com `tasks` que calcule, em paralelo, o quicksort de um vetor, usando `task` para os dois ramos recursivos e `taskwait` para sincronizar antes do retorno. (Pode ser pseudocódigo, sem precisar compilar.)

Q38. Explique, em um parágrafo, por que a versão com `tasks` do Fibonacci pode ficar **mais lenta** que a versão sequencial se você não usar `if` ou `final`.

GABARITO COMENTADO

Parte 1

R1. Sem `single` : 4 vezes (cada thread executa o bloco inteiro). Com `single` : 1 vez (só a primeira thread a chegar executa; as outras esperam na barreira implícita do `single`).

R2. `master` : sempre a thread 0, sem barreira (as demais não esperam). `single` : qualquer thread que chegar primeiro, com barreira implícita (as demais esperam terminar).

R3.

```
#pragma omp parallel num_threads(4) reduction(+:soma)
{
    #pragma omp for
    for (int i = 0; i < N; i++) {
        soma += A[i];
    }
}
```

(ou simplesmente `#pragma omp parallel for reduction(+:soma)` , forma combinada)

Parte 2

R4. `schedule(static,2)` = blocos de 2 em rodízio.

- Thread 0: `i = 0,1, 8,9`
- Thread 1: `i = 2,3, 10,11`
- Thread 2: `i = 4,5, 12,13`
- Thread 3: `i = 6,7, 14,15`

R5. Porque a divisão em `dynamic` é decidida **em tempo de execução**: cada thread pega o próximo chunk disponível assim que fica livre, e isso depende da velocidade real de execução de cada thread naquele momento específico — não é algo determinístico como em `static` .

R6.

Schedule	Decisão tomada quando?	Melhor cenário
static	Antes de rodar (compile/início)	Iterações com custo uniforme
dynamic	Durante a execução	Iterações com custo desigual/imprevisível
guided	Híbrido, blocos decrescentes	Meio-termo entre os dois

R7. `dynamic` (ou `guided`) — já que o custo é desigual entre a primeira e a segunda metade, um `static` deixaria as threads que pegaram a parte lenta muito mais ocupadas

que as outras, criando desbalanceamento. `dynamic` permite que threads que terminam rápido (a parte fácil) peguem mais trabalho da parte difícil.

Parte 3

R8. `shared` (exceto a variável de controle de um `for` paralelizado, que é `private` automaticamente).

R9. É imprevisível — condição de corrida. `x = x + tid` envolve ler, somar e escrever separadamente; múltiplas threads podem ler o mesmo valor "antigo" antes de qualquer uma escrever de volta, perdendo atualizações. O valor final de `x` não corresponde à soma esperada de todos os `tid`.

R10. Dentro da região, cada thread tem sua própria cópia de `x`, começando **indefinida** (lixo de memória) — não herda o valor 100. Fora da região, `x` continua valendo 100 (inalterado), pois `private` não propaga nenhuma alteração de volta.

R11. `private`: cópia isolada, sem valor inicial herdado (indefinida). `firstprivate`: cópia isolada, mas inicializada com o valor que a variável tinha antes da região começar.

R12. `temp_fatorial` está com escopo incorreto (deveria ser `private`, mas por padrão é `shared`, causando condição de corrida entre iterações concorrentes). Correção: `#pragma omp parallel for num_threads(4) private(temp_fatorial)`. (`j` também precisa ser `private` — ou já é automaticamente se declarado dentro do laço `for`.)

Parte 4

R13.

- a) `atomic read`
- b) `atomic write`
- c) `atomic update`
- d) `atomic capture`
- e) Não pode ser um único atomic — são 3 operações relacionadas em sequência; precisa de `critical`.

R14. Porque `atomic` protege apenas **uma única expressão/operação de memória**, não múltiplas instruções relacionadas em bloco.

R15. "Use `atomic` quando a operação é simples (uma leitura/escrita/atualização de uma variável escalar), e use `critical` quando o bloco protegido tem múltiplas instruções ou lógica mais complexa."

R16. Não é ideal — `critical` sem nome usa um cadeado **global único**, então mesmo protegendo variáveis não-relacionadas (`a` e `b`), as threads competem pelo mesmo cadeado desnecessariamente, serializando operações que poderiam ser paralelas.

Melhoria: usar `critical(nome_a)` e `critical(nome_b)` com nomes distintos (ou trocar por `atomic`, já que são incrementos simples).

R17. `barrier` força todas as threads do time a esperarem, nesse ponto, até que **todas** cheguem antes de qualquer uma prosseguir. Situação de uso explícito: quando você tem dois laços com `nowait`, mas o segundo depende de um resultado escrito pelo primeiro — sem barreira explícita, uma thread rápida pode ler dados que outra ainda não escreveu.

R18. A linha (1) ("A: %d") pode sair em qualquer ordem entre as threads — sem restrição. A linha (2), dentro do bloco `ordered`, sempre sai na sequência 0,1,2...9, respeitando a ordem lógica do laço original.

Parte 5

R19. Porque só existem 3 valores possíveis de `i` (laço externo) para dividir entre 4 threads — uma fica sem receber nenhuma fatia. Solução: `collapse(2)`, que "achata" os dois laços ($3 \times 10 = 30$ iterações) num espaço único, permitindo distribuir entre todas as 4 threads.

R20. É **perigoso**. O segundo laço lê `a[i]` (`b[i] = a[i] + 1`), que é escrito pelo primeiro laço. Sem a barreira (removida pelo `nowait`), uma thread pode começar o segundo laço antes que outra thread termine de escrever a posição correspondente de `a` no primeiro laço, causando leitura de dado desatualizado.

R21.

```
#pragma omp parallel for reduction(+:soma) private(x)
for (long i = 0; i < num_passos; i++) {
    x = (i + 0.5) * passo;
    soma = soma + 4.0 / (1.0 + x*x);
}
```

Parte 6

R22. 4 tasks (uma por thread, já que todas as 4 threads encontram a linha `#pragma omp task` de forma independente, sem `single`).

R23.

```
#pragma omp parallel num_threads(4)
#pragma omp single
{
    #pragma omp task
    printf("oi\n");
}
```

R24.

- `g` : shared (global)
- `a` : shared (herdou `shared(a)` do `parallel`)
- `b` : `firstprivate` (local, declarada fora, sem cláusula explícita → padrão da task)
- `c` : `firstprivate` (local, declarada dentro do `parallel` mas fora da task → padrão da task)
- `d` : `private` (declarada dentro do próprio bloco `task`)

R25. Porque, se fossem `firstprivate` (o padrão), cada task teria sua **própria cópia** de `x / y` , e o resultado calculado dentro da task nunca seria "visto" pela função externa depois do `taskwait` — precisa ser `shared` para que a escrita dentro da task afete a mesma posição de memória que a função lê depois.

R26. `taskwait` : espera só as tarefas-filhas **diretas**. `taskgroup` : espera as filhas **e toda a descendência** (netas, bisnetas...). No exemplo dado, com `taskwait` (não `taskgroup`), sim — "Mensagem final" **pode** aparecer antes de "Neto terminou", porque o `taskwait` só espera o "Filho" (filho direto), não o "Neto" (que é filho do filho, ou seja, neto).

R27. `barrier` : afeta **todas as threads do time**, esperando tanto threads quanto tarefas pendentes na região. `taskwait` : afeta apenas a tarefa/thread que o encontrou, esperando somente suas próprias tarefas-filhas diretas — não sincroniza outras threads do time que não passaram por ali.

R28. Quando `n <= 20` , a task (e toda sua árvore de descendentes daquele ponto em diante) deixa de gerar tasks reais — vira execução sequencial direta. Isso evita o overhead de criar/agendar milhares de tasks minúsculas (como calcular `fib` de números pequenos), que custaria mais caro que o trabalho em si.

R29. `if(expr)` decide task por task, isoladamente — cada nível da recursão reavalia sua própria condição. `final(expr)` , quando verdadeira, propaga a decisão de "não gerar mais tasks" para **toda a subárvore abaixo**, sem reavaliação nos níveis seguintes.

Parte 7

R30.

```
#pragma omp task depend(out: x)
{ x = 10; }

#pragma omp task depend(in: x) depend(out: y)
{ y = x * 2; }

#pragma omp task depend(in: x) depend(out: z)
{ z = x + 5; }

#pragma omp task depend(in: y, z)
{ resultado = y + z; }
```

R31. Sim — ambas só **leem** `x` (dependência `in`), e como leitura simultânea não causa condição de corrida, elas podem rodar em paralelo entre si; ambas só esperam T1 (que produz `x`).

R32. T2 passaria a ser tratada também como "escritora" de `x` (por causa do `inout`). Como T3 vem depois de T2 na ordem de criação e ambas mexem em `x`, T3 passaria a **depende de T2 também**, perdendo a capacidade de rodar em paralelo com ela.

R33. `grainsize(n)`: cada bloco/task terá entre `n` e `2n` iterações (o número de tasks resultante é decidido pelo runtime). `num_tasks(n)`: cria exatamente `n` tasks, dividindo o total de iterações entre elas.

R34. `taskloop` cria uma região `taskgroup` implícita — espera automaticamente todas as suas tasks internas terminarem antes de prosseguir. `nogroup` remove essa barreira implícita, exigindo sincronização manual (`taskwait`) se necessário.

R35. Porque cada bloco de iterações vira uma task independente, que pode ser executada por qualquer thread, a qualquer momento — o runtime não garante ordem de execução entre tasks, só garante que, ao final (barreira implícita ou explícita), todas terão sido executadas e escrito seus resultados corretamente nas posições certas do vetor.

Parte 8

R36. Erros: (1) faltou a diretiva de paralelização no laço; (2) `x` precisa ser `private`, `soma` precisa de `reduction(+:soma)`.

```
#pragma omp parallel for reduction(+:soma) private(x)
for (long i = 0; i < num_passos; i++) {
    x = (i + 0.5) * passo;
    soma = soma + 4.0 / (1.0 + x*x);
}
```

R37.

```
void quicksort_par(int *a, int lo, int hi) {
    if (lo < hi) {
        int p = particiona(a, lo, hi);
        #pragma omp task firstprivate(a, lo, p)
        quicksort_par(a, lo, p - 1);
        #pragma omp task firstprivate(a, p, hi)
        quicksort_par(a, p + 1, hi);
        #pragma omp taskwait
    }
}

// chamada inicial:
#pragma omp parallel
```

```
#pragma omp single  
quicksort_par(vetor, 0, n - 1);
```

R38. `fib(n)` gera uma quantidade enorme de chamadas recursivas, e cada uma cria 2 tasks. Quando `n` fica pequeno, o trabalho real de somar dois números é trivial — mas o custo de **criar e agendar** uma task (alocação, gerenciamento no pool, sincronização no `taskwait`) é muito maior que esse trabalho trivial. O overhead de gerenciamento das tasks passa a dominar o tempo total, tornando a versão paralela mais lenta que a sequencial pura.